

提高组 CSP-S 2024 初赛模拟卷 3

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下面的代码属于哪个算法的核心代码？（ ）

```
long long c[20][20];
c[0][0]=1;
for(int i=1;i<=n;i++){
    c[i][0]=c[i][i]=1;
    for(int j=1;j<i;j++){
        c[i][j]=c[i-1][j-1]+c[i-1][j]
    }
}
```

A. 0-1 背包 B. 杨辉三角形 C. 完全背包 D. 部分背包

2. NOI Linux 2.0 中缺省指定编译的 C++ 版本是（ ）。

A. C++03 B. C++17 C. C++11 D. C++14

3. 下列属于视频文件格式的是（ ）。

A. PNG B. AVI C. JPEG D. BMP

4. 以下不属于常见的平衡树的是（ ）。

A. AVL 树 B. 伸展树 C. 三叉树 D. 红黑树

5. 字符串 "dededfed" 和字符串 "defed" 的最长公共子串是（ ）。

A. fed B. defed C. def D. de

6. 下面哪个操作不属于二叉搜索树的常见操作？（ ）

A. 查找 B. 插入 C. 删除 D. 交换排序

7. 具有 m 个顶点、 n 条边的有向图采用邻接表存储结构，进行深度优先遍历运算的时间复杂度是（ ）。

A. $O(m)$ B. $O(n)$ C. $O(m+n)$ D. $O(nm)$

8. 将关键码集合 {100, 300, 800, 700, 500, 900} 逐一保存在一张长度为 100 的哈希表中。选取哈希函数为 $\text{Hash}(\text{key}) = \text{key} / 100$ ，则 500 保存在表中的位置应该是 ()。
- A. 5 B. 6 C. 7 D. 8
9. 设树 T 的高度为 4，其中度为 1、2、3 和 4 的结点个数分别为 4、2、1、1，则 T 中的叶子结点数为 ()。
- A. 5 B. 6 C. 7 D. 8
10. 在 C++ 程序中执行 `cout << (9>8>7>6)` 会输出什么结果? ()
- A. 1 B. 0 C. true D. false
11. 已知 240 个一模一样的小球里面有一个次品，比正常球轻一些。现在有一架天平（每一端都可以放下 240 个小球），请问最少称重几次可以找出这个次品小球? ()
- A. 6 B. 4 C. 5 D. 7
12. 正六边形的中心和顶点共 7 个点，以其中 3 个点为顶点的三角形共有 () 个。
- A. 30 B. 35 C. 32 D. 28
13. 下面的 C++ 代码运行后，输出的是 ()。
- ```
#include<bits/stdc++.h>
using namespace std;
int main()
{
 int a[100]={1,2};
 string s="CSP-S";
 cout<<s[a[2]]<<endl;
 return 0;
}
```
- A. C                      B. S                      C. P                      D. CSP-S
14. 由数字 0, 1, 2, 3, 4, 5 组成没有重复数字的六位数，其中个位数字小于十位数字的共有 ( ) 个。
- A. 210                      B. 300                      C. 464                      D. 600

15. 已知元素(7, 25, 14, 87, 51, 90, 5, 19, 20), 问这些元素以怎样的顺序进入栈, 才能使出栈的顺序满足: 7 在 51 的前面; 90 在 87 的后面; 20 在 14 的后面; 25 在 5 的前面; 19 在 90 的后面? ( )
- A. 20, 5, 7, 51, 90, 25, 14, 19, 87  
B. 51, 5, 19, 20, 14, 8, 87, 90, 25  
C. 19, 20, 90, 7, 5, 25, 51, 14, 87  
D. 5, 25, 51, 7, 20, 19, 90, 87, 14

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <iostream>
02 using namespace std;
03
04 const int mod = 1e9+7;
05 const int N = 1010, C = 10010;
06 int nn, cc;
07 long long a[2][C], b[2][C];
08
09 int main() {
10 cin >> nn >> cc;
11 a[1][0] = 1; b[1][0] = 1;
12 for(int n = 2; n <= nn; n++) {
13 for(int c=min(cc,(n-1)*(n-2)/2)+1;
14 c<=min(cc,n*(n-1)/2); c++) {
15 b[(n-1) & 1][c] = b[(n-1) & 1][c-1];
16 }
17 a[n & 1][0] = 1; b[n & 1][0] = 1;
18 for (int c = 1; c <= min(cc, n*(n-1)/2); c++) {
19 int en = max(c-n+1, 0);
20 a[n & 1][c] = b[(n-1) & 1][c];
21 if (en > 0) a[n & 1][c] -= b[(n-1) & 1][en-1];
22 a[n & 1][c] %= mod;
23 if (a[n & 1][c] < 0) a[n & 1][c] += mod;
24 b[n & 1][c] = b[n & 1][c-1] + a[n & 1][c];
25 b[n & 1][c] %= mod;
26 }
27 }
```

```
26 }
27 cout << a[nn & 1][cc] << endl;
28 return 0;
29 }
```

### ■ 判断题

16. 输入为"10 1"时, 输出为"10"。 ( )
17. 输入为"4 3"时, 输出为"6"。 ( )
18. 本题与计算逆序对有关。 ( )
19. 第 16 行可以放到第 13 行前面, 程序的运行结果不会改变。 ( )

### ■ 选择题

20. 本程序的时间复杂度是 ( )。
- A.  $O(n^2)$       B.  $O(n^3)$       C.  $O(n^2 \log n)$       D.  $O(n^4)$
21. 数组 a、b 的调用经常有"&1"的操作, 这是为了 ( )。
- A. 没什么意义, 干扰读题      B. 用位运算快速减一
- C. 快速取二进制尾数      D. &运算符用于获取存储地址

### (2)

```
01 #include <iostream>
02 #include <iomanip>
03 #include <math.h>
04 using namespace std;
05 pair<long long, long long> calc(int n, long long m){
06 if (n==0) return make_pair(0,0);
07 long long len = 1ll << (n-1), cnt = 1ll << (2*n-2);
08 pair<long long, long long> pos = calc(n-1, m % cnt);
09 long long x = pos.first, y = pos.second;
10 long long z = m / cnt;
11 if (z==0) return make_pair(y, x);
12 if (z==1) return make_pair(x, y+len);
13 if (z==2) return make_pair(x+len, y+len);
14 if (z==3) return make_pair(2*len-y-1, len-x-1);
15 }
16 int main() {
17 int n, p, q;
18 cin >> n >> p >> q;
```

```
19 cout << setprecision(3)
20 << sqrt(pow(1.0*calc(n, p).first-calc(n,q).first,2.0)
21 + pow(1.0*calc(n,p).second-calc(n,q).second,2.0))
22 << endl;
23 return 0;
24 }
```

### ■ 判断题

22. 本程序编译会报 warning。 ( )
23. 若想程序不出错,  $p$ 、 $q$  不能超过  $2^{2^n}$ 。 ( )
24. 输入数据符合要求时, 本程序的时间复杂度是  $O(\max\{N, \log M\})$ 。 ( )

### ■ 选择题

25. 若输入为 "2 0 10", 输出为 ( )。
- A. 4.243                      B. 3.00                      C. 4.24                      D. 3.606
26. 当  $|p-q|=1$  时, 不论输入的  $n$  是多少, 只要  $p$ 、 $q$  在符合要求的范围内, 则输出为 ( )。
- A. 1                      B. 2                      C. -1                      D. 0
27. 对同一个  $n$ , 共有几种输入, 可以让输出达到最大? ( )
- A. 2                      B. 4                      C. 1                      D. 8

### (3)

```
01 #include <iostream>
02 using namespace std;
03
04 const int N = 500010;
05 int n;
06 long long ans, mod = 1000000007;
07 long long a[N], b[N], A[N], B[N], sa[N], sb[N];
08
09 int main() {
10 cin >> n;
11 for (int i = 1; i <= n; i++) {
12 cin >> a[i];
13 A[i] = A[i-1] + a[i];
```

```
14 A[i] %= mod;
15 sa[i] = sa[i-1] + A[i];
16 sa[i] %= mod;
17 }
18 for (int i = 1; i <= n; i++) {
19 cin >> b[i];
20 B[i] = B[i-1] + b[i];
21 B[i] %= mod;
22 sb[i] = sb[i-1] + B[i];
23 sb[i] %= mod;
24 }
25 long long t;
26 for (int i = 0; i <= n; i++) {
27 t = A[i];
28 t *= B[i]; t %= mod;
29 t *= n; t %= mod;
30 ans += t;
31 ans %= mod;
32 }
33 for (int i = 0; i < n; i++) {
34 t = A[i] * (sb[n] - sb[i]);
35 t %= mod;
36 ans -= t;
37 if (ans < 0) ans += mod;
38 t = B[i] * (sa[n] - sa[i]);
39 t %= mod;
40 ans -= t;
41 ans %= mod;
42 if (ans < 0) ans += mod;
43 }
44 cout << ans << endl;
45 return 0;
46 }
```

#### ■ 判断题

28. 因为有第 39 行, 所以若去掉第 35 行, 程序结果不受影响。 ( )
29. 去掉第 37 行, 程序结果不受影响。 ( )
30. 第 34 行有溢出风险, 必须按照第 27~28 行的方式来写。 ( )



■ 选择题

31. 若输入 3 2 3 4 3 4 5, 则输出为 ( )。

- A. 243                      B. 244                      C. 253                      D. 254

32. 本题实际在计算 ( )。

- A.  $\sum_{r=1}^n \sum_{l=r}^n (\sum_{i=l}^r a_i * \sum_{i=l}^r b_i)$                       B.  $\sum_{l=1}^n \sum_{r=l}^n (\sum_{i=l}^r a_i * b_i)$
- C.  $\sum_{l=1}^n \sum_{r=1}^n (\sum_{i=l}^r a_i * \sum_{i=l}^r b_i)$                       D.  $\sum_{l=1}^n \sum_{r=l}^n (\sum_{i=l}^r a_i * \sum_{i=l}^r b_i)$

33. (4 分) 本代码将原问题的时间复杂度从  $O(N^3)$  优化到  $O(N)$ 。如果希望继续优化, 应该 ( )。

- A. 没有太大优化空间  
B. 减少取模运算  
C. 将 long long 改为 int, 以提高处理速度  
D. 做更多的预处理

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 有一个字符串 S, 最开始其中只有一个字符 a, 之后你要对这个字符串进行若干次操作, 每次将其中每一个字符 c 替换成某个字符串 s (例如对于字符串 ball, 将其中的 l 替换为 na 后将得到 banana)。现在给定  $l, r$ , 你需要输出变换完毕后, S 的第  $l$  个字符到第  $r$  个字符对应的子串是什么。

算法是逆序建树, 从最后一次操作开始, 将字符替换的规则视为树的连线, 叶子结点为最后被替换的字母。建树完毕后, 再根据  $l, r$  的数值, 在树上 DFS 遍历并输出。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 #define ll long long
04 const int NN = 2e5+10;
05 char c[NN]; string s[NN];
06 int first[26]; ll tot[NN], tf[26];
07 vector<ll> cnt[NN]; vector<int> nxt[NN];
08
09 int num(char c){
10 return ①;
11 }
```

```
12 ll search(int cur, ll l, ll r, ll acc){
13 for(int i = 0; i < cnt[cur].size(); i++){
14 if(acc >= r) break;
15 if(acc + cnt[cur][i] >= l){
16 if(!nxt[cur][i]){
17 cout << s[cur][i];
18 acc++;
19 }
20 else
21 ②;
22 }
23 else
24 acc += cnt[cur][i];
25 }
26 return acc;
27 }
28 int main(){
29 ios_base::sync_with_stdio(false);
30 cin.tie(NULL);
31 ll l, r; int n;
32 cin >> l >> r >> n;
33 for(int i = 1; i <= n; i++){
34 cin >> c[i] >> s[i];
35 }
36 for(int i = n; i > 0; i--){
37 for(int j = 0; j < s[i].size(); j++){
38 ③;
39 first[num(c[i])] = i;
40 }
41 for(int i = 0; i < 26; i++) tf[i] = 1;
42 for(int i = n; i > 0; i--){
43 for(int j = 0; j < s[i].size(); j++){
44 ④;
45 cnt[i].push_back(tf[k]);
46 tot[i] += tf[k];
47 if(tot[i] > r) break;
48 }
49 tf[num(c[i])] = tot[i];
50 }
51 ⑤;
52 }
```



34. ①处应填 ( )。

- A. `c-a`                      B. `c-'a'`                      C. `c-'a'+1`                      D. `'c'-'a'`

35. ②处应填 ( )。

- A. `acc = search(acc+nxt[cur][i], l, r, acc)`  
B. `acc = search(nxt[cur][i], l-cnt[cur][i], r, acc)`  
C. `acc = search(nxt[cur][i], l, r, acc)`  
D. `acc = search(nxt[cur][i], l, r, acc+cnt[cur][i])`

36. ③处应填 ( )。

- A. `nxt[i].push_back(first[num(s[i][j])])`  
B. `nxt[i].push_back(num(s[i][j]))`  
C. `cnt[i].push_back(num(s[i][j]))`  
D. `cnt[i].push_back(first[num(s[i][j])])`

37. ④处应填 ( )。

- A. `int k = 0`                      B. `int k = num(c[i])`  
C. `int k = num(s[i][j])`                      D. `int k = first[num(s[i][j])]`

38. ⑤处应填 ( )。

- A. `search(first[1], l, r, nxt[0])`  
B. `search(first[1], l, r, 0)`  
C. `search(0, l, r, 1)`  
D. `search(1, l, r, 0)`

(2) 给定函数  $h(x)$ ,  $x \in Z$ ,  $1 \leq x \leq n$ , 定义  $\text{seepairs}(h)$  为这样的无序数对  $(i, j)$  的个数:  $i < j$ , 且连接  $(i, h(i))$  与  $(j, h(j))$  的线段不低于  $(k, h(k))$ , 其中  $i < k < j$ 。对  $h$  函数进行  $Q$  次调整, 每次把  $h(x)$  调整为  $h(x) + y$ ,  $y > 0$ 。求每次调整之后的  $\text{seepairs}(h)$ 。

数据规模:  $n, Q \leq 2000$ , 初始  $h(x)$  和调整后的  $h(x)$  都不超过  $10^9$ 。

本代码使用 `set` 存储, 初始化每个  $h(i)$  能“看到”的点的数量  $\text{see}(i)$ , 则对  $\text{see}(i)$  求和即为  $\text{seepairs}(h)$ , 每次调整  $h(x)$  之后, 因为  $h(x)$  只增不减, 所以  $\text{see}(x)$  只增不减。对于  $i > x$ ,  $\text{see}(i)$  不变, 对于  $i < x$ ,  $\text{see}(i)$  只减不增, 因此把  $\text{see}(1)$  至  $\text{see}(x)$  调整之后再求和, 即为调整后的  $\text{seepairs}(h)$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 const int N = 2010;
05 int n, q, x, y; long long h[N];
06 set<int> see[N];
07 typedef set<int>::iterator iter;
08
09 bool small(int i, int k, int j) {
10 return ①;
11 }
12 int main() {
13 cin >> n;
14 for (int i = 1; i <= n; i++)
15 cin >> h[i];
16 for (int i = 1; i < n; i++) {
17 see[i].insert(i+1);
18 for (int j = i+2; j <= n; j++) {
19 ②;
20 see[i].insert(j);
21 }
22 }
23 cin >> q;
24 for (int qq = 1; qq <= q; qq++) {
25 cin >> x >> y;
26 h[x] += y;
27 for (int i = 1; i < x; i++) {
28 iter it = ③;
29 if (it == see[i].end()) {
30 --it;
31 if (small(i, *it, x)) see[i].insert(x);
32 } else {
33 bool ins = false;
34 if (*it == x) {
35 ins = true;
36 it ++;
37 } else {
38 --it;
39 if (small(i, *it, x)) {
40 see[i].insert(x);
```

```
41 ins = true;
42 it++;
43 ④;
44 }
45 }
46 if (ins)
47 while (⑤) {
48 it = see[i].erase(it);
49 }
50 }
51 }
52 if (x < n) {
53 see[x].clear();
54 see[x].insert(x+1);
55 for (int j = x+2; j <= n; j++) {
56 if (small(x, *see[x].rbegin(), j))
57 see[x].insert(j);
58 }
59 }
60 int ans = 0;
61 for (int i = 1; i < n; i++)
62 ans += see[i].size();
63 cout << ans << endl;
64 }
65 return 0;
66 }
```

39. ①处应填 ( )。

- A.  $(h[k] - h[i]) * (j - k) < (h[j] - h[k]) * (k - i)$
- B.  $(h[k] - h[i]) * (j - k) \leq (h[j] - h[k]) * (i - k)$
- C.  $(h[k] - h[i]) * (j - i) < (h[j] - h[i]) * (k - i)$
- D.  $(h[k] - h[i]) * (j - i) \leq (h[j] - h[i]) * (k - i)$

40. ②处应填 ( )。

- A. `if (small(i, *see[i].rbegin(), j))`
- B. `if (small(i, *see[i].rbegin(), j-1))`
- C. `if (small(i, *see[i].end(), j))`
- D. `if (small(I, *see[i].end()-1, j-1))`

41. ③处应填 ( )。
- A. `see[i].upper_bound(x)`
  - B. `see[i].lower_bound(x)`
  - C. `see[i].lower_bound(x)+1`
  - D. `see[i].upper_bound(x+1)`
42. ④处应填 ( )。
- A. `it--`
  - B. `it++`
  - C. `//无内容`
  - D. `}`
43. ⑤处应填 ( )。
- A. `it != see[i].end()-1 && !small(i, x, *(it))`
  - B. `it != see[i].end()-1 && !small(i, x, it)`
  - C. `it != see[i].end() && !small(i, x, *(it))`
  - D. `it != see[i].end() && !small(i, x, it)`